Everything but the Software: Supplementary Case Evidence and Definitions

Juan Manuel Servera Bondroit

Draft of 1 November 2026

This companion retains the detailed rival-explanation register, candidate-mechanism audits and historical measurement definitions. It adds no experimental results or independent human corroboration. The five-arm prospective protocol is in the main article and current researcher kit. The consolidation prompt reproduced here is the historical v4 prompt, not the amended experiment's approved execution source.

1 Background

1.1 Software that only grows

That software decays when it only accumulates features is one of the oldest observations in software engineering. Lehman's laws of software evolution hold that a program in real-world use must keep changing, and that its complexity grows as it evolves unless work is done to maintain or reduce it [44]; a later formulation adds that its quality will appear to decline unless it is rigorously maintained and adapted [43]. Parnas called the result software aging, and blamed in part the changes made by people who do not understand the original design [60]. Eick and colleagues found mixed but, on the whole, persuasive evidence of code decay in fifteen-plus years of change data from a large telephone-switching system [26]. Cunningham's technical-debt metaphor framed the cost of not consolidating as interest paid on every later change [22], and Fowler's design stamina hypothesis holds that without attention to design, adding features eventually becomes slower than it would have been with it [27]. Studies have since put numbers on the interest: across 39 proprietary code bases, low-quality files had fifteen times more defects and their issues took 124% longer to resolve in a CodeScene-associated vendor study [69]; 43 developers self-reported losing about 23% of their working time to technical debt in a longitudinal study [14]. These are different outcomes and populations.

The evidence is not universal. Increasing complexity and declining quality are among the less strongly supported of Lehman's laws, and some systems have grown without the predicted decay [34]. Several findings above are observational, and their estimates belong to the studied settings, not every code base.

The engineering intuition often called *software entropy* is that keeping a changing system coherent requires counter-work: removing obsolete paths, restoring boundaries and revisiting assumptions. Software does not deteriorate simply because time passes, and thermodynamic entropy supplies no measure of its quality. Here the measurable quantities are losses of required behaviour, excess structural burden and the effort needed for a later change.

1.2 What carries from one feature to the next?

Feature accumulation is the addition of capabilities and interactions; churn is the editing activity that implements, replaces or removes them. Neither is itself deterioration. A *regression* loses previously working behaviour that is still required. A never-implemented feature is instead

an acquisition failure. Persistence means that the same fault remains unresolved; propagation means that a later, distinct fault or failure can be traced to reuse of the earlier one. *Compounding* is the stronger claim that the earlier error increases subsequent error risk or change burden.

These distinctions matter because an agent inherits more than a code diff. The repository supplies implementations, tests, interfaces and deployment assumptions. Instructions, skills and notes supply an account of how to work on them. A mistaken rule can become code, then a test expectation or an instruction, and finally the precedent used for another feature. The stored record need not be correct to become authoritative in the next session. Conversely, an inherited implementation can be sound even when it differs from a human reference. The question is what subsequent work can achieve from that state.

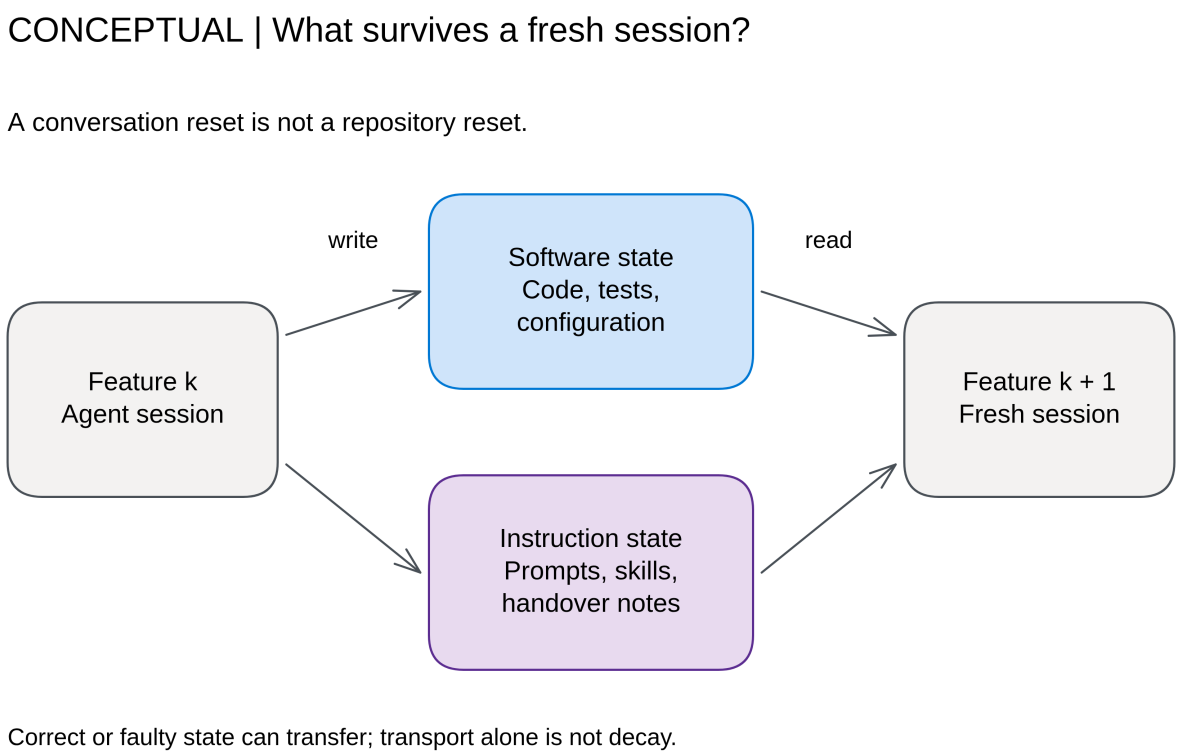


Figure 1: Conceptual state transfer between feature sessions. A fresh conversation does not reset inherited software or instruction state. Solid arrows denote possible read/write paths, not measured error propagation. Both channels can carry correct decisions as well as faults; their presence alone is not evidence of decay.

Figure 1 separates the two persistent channels from conversation history. It is an explanatory model, not an observed trajectory or a claim that every tool retains the same material.

1.3 Compounding within and between features

Errors carried in conversation. An earlier answer changes the context from which a model produces the next one. Experiments on this pathway distinguish dependence on an erroneous history from the difficulty of a long task.

Zhang et al.'s earlier *How Language Model Hallucinations Can Snowball* shows why consistency is not enough. On three question-answering datasets, ChatGPT and GPT-4 produce false justifications after wrong answers, yet often reject the individual false claims when questioned in a separate session. Their explanation is over-commitment to the earlier answer rather than only a lack of knowledge. The inspected original experiments concern question answering in two

2023 models; the later ICML publication expands the model coverage [74, original preprint, Secs. 3.1–3.4].

Sinha et al. test the distinction directly in a controlled execution task. Models receive both the lookup dictionary and the plan, then maintain a running sum. The authors vary the fraction of incorrect answers in artificial prior histories and measure the accuracy of subsequent state updates, not merely whether the final total remains correct. More erroneous history reduces later accuracy beyond the effect of context length. This is counterfactual evidence of *self-conditioning*: earlier mistakes can increase the chance of new mistakes. Scaling non-thinking models does not remove the effect, but Qwen3 models with thinking enabled avoid it in this setup. The result identifies a dependence on model configuration as well as erroneous history [66, Secs. 2.2–3, Figs. 5–6].

Laban et al. supply complementary conversational evidence. Across six generation tasks, including Python and SQL, they report a 39% average relative performance drop when instructions arrive through underspecified multi-turn conversation rather than all at once. A concatenated-instruction control largely preserves performance, weakening information loss from rephrasing as the explanation. Their analyses associate premature answers with lower success and describe reliance on earlier incorrect attempts and increasingly bloated solutions. These are simulated conversations and diagnostic analyses, not a controlled estimate of the cost of a particular inherited coding defect; the 39% is not a percentage-point loss or a repository defect rate [41, arXiv v1, Secs. 3.3 and 6.1, App. F.1–F.2].

Errors carried in the repository. Pan et al.'s *SWE-Maintain* preprint tests the pathway most directly relevant to successive feature work. It reconstructs 100 three-event maintenance episodes from 50 Python repositories and runs each later task from two starting points: the developers' preceding patches and the agent's own preceding changes. Task, agent and tests are held fixed, and every event uses a fresh session: only the repository carries the earlier work forward. Across 1,996 paired later events, agent history lowers new-task resolution by 5.6 percentage points (95% interval $[-7.2, -4.1]$). Requiring earlier tests to remain passing widens the gap to 14.0 points. The unweighted mean new-task gap grows from 4.0 points at the second event to 7.2 at the third, consistent with an accumulating downstream cost, though three-event episodes cannot establish a general long-horizon decay law [59, Secs. 2.5 and 4.1, Table 2].

Their `python_unidiff` example makes the proposed propagation concrete: an agent splits a string into lines, losing line terminators; the next task reuses that logic in a constructor, while developer history succeeds. The framing appears in Figure 1's caption and the detailed failure in Section 5.1 and Figure 5 [59]. Their `parse_crontab` example exposes a less visible dependency: an agent's local Python 3 fix passes the current and next tests, but later code relies on module-wide compatibility aliases that it never supplied. The later compatibility failure is discussed in Section 5.1 and Figure 5 [59]. Among the 619 later events whose inherited code passes earlier tests and whose next task is not already solved, the new-task-only difference is -1.1 points (95% interval $[-3.8, +1.6]$), and the event-resolution difference is $+2.7$ points (95% interval $[-0.5, +6.1]$), with no detected disadvantage. This is a selected clean-history subset, not a randomised repair intervention, and it does not prove equivalent maintainability under all changes. It points more directly to inherited behavioural failures than to structural complexity as the main observed channel [59, Sec. 4.2, Table 3; Sec. 5.1, Fig. 5].

Figure 2 shows the published event-resolution contrasts, including the clean-history cohorts. Event resolution requires the new task and all earlier non-superseded tests to pass. Its larger gap therefore includes retained failures as well as difficulty solving the new task. The current-task-only difference of -5.6 points is important for the cross-feature hypothesis precisely because it does not count earlier failing tests against success on the new task.

The conversation and repository studies therefore locate two different transmission pathways. Resetting context addresses the first, but not faulty code retained for the next feature.

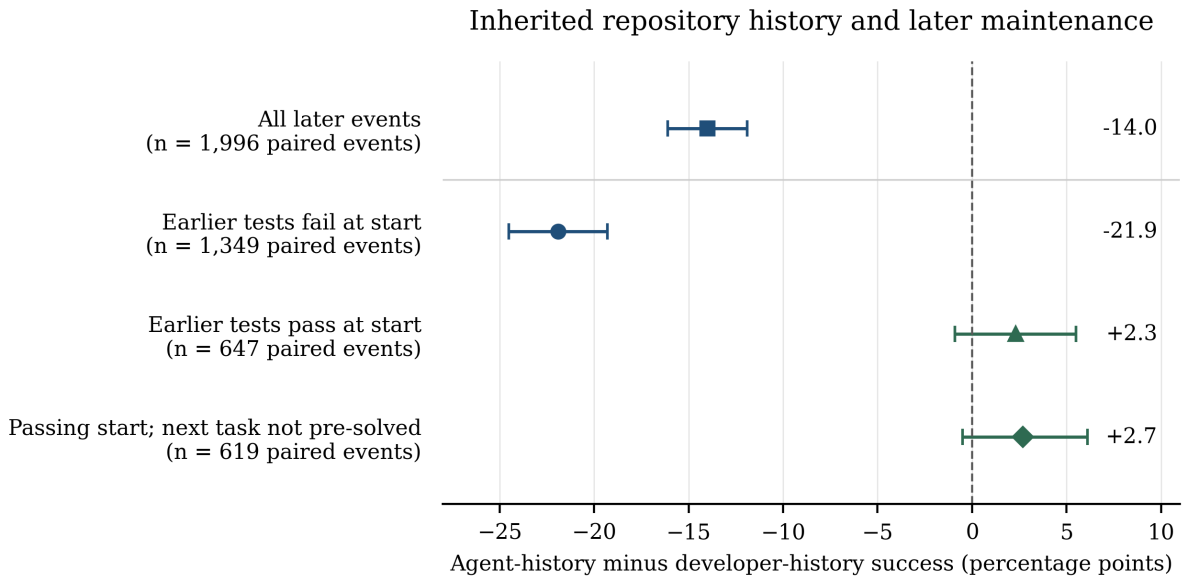


Figure 2: Later maintenance from two repository histories. Points and 95% repository-clustered bootstrap intervals are transcribed from Pan et al., Table 3 [59]. The outcome is event resolution: current and all earlier non-superseded hidden tests pass. Negative differences favour developer history. The 1,996 paired events come from 100 three-event episodes in 50 Python repositories, evaluated with five systems and two models. Subgroups overlap and are selected by the agent-history starting state; the last excludes pre-solved and unevaluable tasks. They are not randomised repair conditions.

SlopCodeBench and SWE-CI extend the evaluation horizon to iterative structural erosion and behavioural regressions [20, 58]. A maintainability study using late-2024 coding assistants found no significant downstream difference; only its second phase was randomised [16]. This is assistant-era evidence, not a test of the current autonomous agents. The emerging question is which inherited failures survive, how they affect later work, and which interventions prevent or repair them.

Output and review capacity. Understanding changes is a major challenge of code review; habituation, fragmented context and uneven retrieval from long contexts may limit scrutiny as output grows [2, 18, 35, 46, 73]. Faster feature delivery also shortens the time available to discover that an earlier assumption has become the foundation of another change. Further empirical and practitioner context is retained in the ancillary supplement.

1.4 Corrective instructions as maintenance debt

A failed agent action often invites a new instruction: run this check, never edit that file, remember this exception. GitHub's account of more than 2,500 agent-definition files recommends starting small, testing and adding detail when an agent makes mistakes [57]. The resulting guidance can preserve useful lessons, but it also becomes another maintained system. Chatlatanagulchai et al. document ongoing maintenance of this material [19, Secs. 4.2.2–4.2.3]. Here *instruction debt* denotes the avoidable burden of obsolete, duplicated, conflicting or poorly routed directives in prompts, skills and handover notes. This extends the maintenance question from the product to the machinery that changes it.

Repository histories show that this material evolves. Mohsenimofidi et al. found no post-creation change in 77 of 155 instruction files. In the ten most frequently changed files, however, they classified 169 commits: 78 added instructions, 59 modified them and 23 removed them.

These are three of nine categories, with 242 assignments in total, rather than a partition of the 169 commits [53, Sec. 4.3, Table 2]. Chatlatanagulchai et al.'s manual analysis of 100 post-adoption commits produced 78 addition, 32 modification and 19 deletion labels; addition labels occurred across 66 distinct commits. Figure 3 replots the label counts [19, Secs. 4.2.2–4.2.3]. These histories contain accumulation and pruning, rather than a uniform march towards longer files. The authors associate their day-scale edit intervals with ongoing project maintenance rather than immediate prompt debugging.

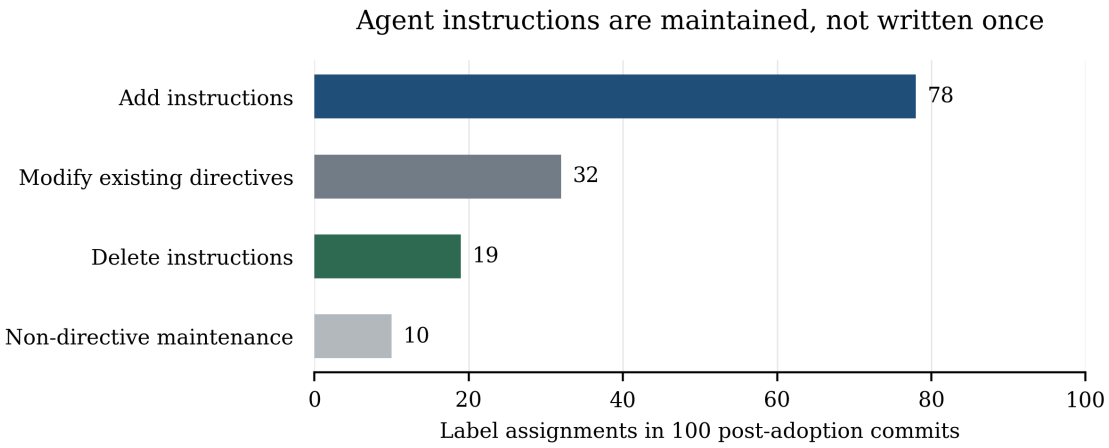


Figure 3: Instruction-file maintenance in 100 randomly sampled post-adoption commits, from Chatlatanagulchai et al., Sections 4.2.2–4.2.3 [19]. Commits can receive several labels; line-level and section-level add/delete labels are combined. The ten non-directive maintenance labels are the remainder of 139 total assignments. The chart counts edit types, not net token growth or effects on agent performance.

Gloaguen et al. test the effect of supplying such context. Across four agent–model pairings on 300 SWE-bench Lite tasks and 138 CTXbench tasks, LLM-generated context files increased average inference cost by 20% and 23%, respectively; task success was slightly lower, by 0.5 and 2 percentage points, without significant differences from the no-file baseline. Developer-written files performed significantly better than LLM-written files on CTXbench ($p = 0.038$). Agents followed the instructions, performed more testing and exploration, and spent more reasoning tokens. Yet file length did not explain success, and generated context helped when other documentation was removed [31, Secs. 4.2–4.4, App. B]. Instruction count and loaded tokens are therefore different exposures from the obligations and behaviours those instructions induce.

Anthropic's context-engineering account describes the practical tension: provide enough information to guide action, but avoid brittle prompt logic and an expanding catalogue of exceptions [63]. Maintaining these instructions means preserving necessary constraints while replacing accumulated exceptions with coherent rules, executable checks and task-specific retrieval. To test the corrective-rule spiral directly would require versioned instructions, actual session inputs and outcomes across successive revisions. The testbed record supplies document growth and rule changes; the consolidation protocol tests a bounded form of handover maintenance by replacing, rather than appending to, its design note.

SkillOpt supplies counterevidence to treating instruction change as inherently harmful. Its first-party account describes bounded edits to skill files, held-out validation and retention of successful versions without changing the target model's weights [52]. Procedural knowledge needs evaluation and maintenance; its length alone tells us little about its quality.

Jiang et al.'s preprint compares workflow memory with skills distilled from the same execution histories. Its paired trajectory analysis associates skills chiefly with more reliable procedures, while also identifying misapplication and insufficient runtime verification [38, Secs. 3.3, 5.1–

5.3]. This qualifies an instruction-debt account: retained guidance can stabilise execution rather than merely burden context. Their retrieval study also separates skill access from task success; in their comparison, lower exact-match access was observed alongside sustained downstream success [38, Sec. 5.5]. Because retrieval diagnostics are run separately from task execution, this comparison does not isolate the effect of reduced skill access. Neither result measures the effects of accumulating repository instructions across feature changes.

1.5 Refactoring, review and evaluation boundaries

Agentic refactoring, review and human correction are already studied. RefactorBench evaluates 100 handcrafted multi-file refactoring tasks across nine Python repositories, not scheduled consolidation between feature additions [30]. Kumar et al.'s ASE 2025 study observed developers collaborating with agents on existing issues: incremental, active collaboration was associated with greater success, not randomly assigned to establish an optimal intervention rate [40]. SWE-Sharp-Bench supplies a reproducible C# issue-resolution benchmark and curation pipeline [49]; reproducible repository-scale evaluation itself is not a contribution claimed here.

Two recent preprints sharpen the evaluation problem. CRBugBench evaluates review agents on 1,000 open-source and 800 proprietary tasks from one company, reporting weaker detection on complex, non-local and proprietary-context changes [72]. It is a 9 October 2026 preprint, not an independently replicated result establishing that review cannot work. *Building to the Test* reports near-perfect behavioural scores for demos while the requested reusable library was unused or absent in seven of twelve oracle-in-the-loop runs. Its unchanged, source-hidden oracle did not check that delivery contract [47]. This is distinct from altering tests, as examined in ImpossibleBench [75]: test integrity and adequate outcome coverage are separate safeguards.

These studies narrow the proposed novelty to the case trace and the specific prospective package comparison, including an unforeseen change request. They also caution against assuming that another reviewer, a stricter check or a high hidden-test score necessarily certifies the intended deliverable. Preprints, observational studies, practitioner accounts and first-party industrial reports have different evidential roles; none substitutes for the planned controlled comparison. AgentRx's first-party release describes evidence-backed diagnosis of a trajectory's first critical failure [51]. It motivates traceable failure localisation. An independent diagnosis of the AMA mechanisms remains outstanding.

1.6 Optimisation with behavioural feedback

Pavlenko et al.'s *Towards an AI Software Factory for Data Systems* offers a complementary case [61]. Hotspan packages optimisation targets from telemetry, analysis and reviews; Darwin explores candidate changes through code–test–debug–reflect cycles. The setting is existing, human-architected systems with stable semantics and measurable objectives such as latency. Reported optimisation gains are evidence against treating high agent throughput as inherently unproductive, not a controlled estimate of end-to-end productivity, general maintainability or future changeability. Their application-work productivity calculation excludes infrastructure and onboarding time, and output enters engineering review. The work establishes a precedent for bounded agent-led correction; it does not isolate consolidation between feature additions or product-goal selection.

1.7 The practitioner conversation

Waseem et al. describe rapid generation alongside architectural inconsistency, security concerns and maintenance overhead [70]. That trade-off is not claimed as new here; the contribution is the case trace and prospective package comparison.

Beck supplies the conceptual bridge: augmented coding retains care for code, complexity and tests; alternating features with consolidation aims to restore options consumed by additions [6,

7, 9]. Understanding and abstraction are distinct from producing more artefacts [8]; learning through working software is developed in Beck's talk [9]. These arguments motivate testing bounded reflective work, not an established remedy or a permanent human–agent boundary. Extended practitioner accounts remain in the ancillary supplement.

Spec-driven development adds structure, but not an automatic outcome oracle. Spec Kit proposes specification, planning, tasks and implementation; Böckeler describes both ignored and over-followed instructions; and a study of practice reports reliability benefits alongside planning and review costs [1, 15, 23].

1.8 Engineering principles behind consolidation

The engineering lineage is more specific than a general call for better code. Beck links small structural changes to test feedback [4, 5]; Gamma, Helm, Johnson and Vlissides describe reusable designs with explicit applicability, consequences and trade-offs [29]. Martin emphasises executable technical feedback [48], while Fowler's design stamina hypothesis connects attention to design with the cost of later features [27]. These ideas motivate the consolidation checklist: locate responsibilities, expose dependencies, preserve contracts and leave a concise account of design choices. They are design rationales, not empirical evidence that the proposed agent intervention is effective. Applying more named patterns is not itself an outcome; retained behaviour and the cost of subsequent change are the proposed tests.

Naur distinguishes a programmer's theory of the system from its documentary record; Brooks locates essential difficulty in the conceptual construct, not its representation [17, 56]. Royce and Gall motivate early experience of working behaviour [28, 64]. XP's short releases and tests [4, 10], Crystal Clear's incremental team practice [21], tracer bullets [37] and Agile's preference for working software [12] make that feedback concrete. Small reversible structural changes supply a consolidation practice [5].

Automation can shift effort from producing to monitoring and intervention, with substantial metacognitive demands [3, 65, 68]. Associations between confidence in AI and reduced scrutiny do not establish causation [42]. Goodhart and Strathern motivate checking completion proxies against their intended outcomes [33, 67].

1.9 Hypervelocity Engineering (HVE Core)

HVE Core is an open-source framework from Microsoft that supplies workflow structure for GitHub Copilot; the author's coordinator provided autonomous worker dispatch. Its Research, Plan, Implement and Review (RPI) workflow leaves dated records under `.copilot-tracking`, with explicit human stops and review. The case used those file-based handovers and its Experiment Designer's Minimum Viable Experiment method [50]; the coordinator supplied the separate autonomous delivery loop. Framework product details are secondary context, retained in the ancillary supplement. The author reports using HVE Core plugin snapshot `plugins-v3.3.106` throughout the case. Its snapshot commit is `2df5013e29ebfd2ce61bcd0cf021d8c8186482eb`, generated from source commit `e9af018fe5ffbfbc72321a8bca21fb987359af08`. Historical use is an author account; the snapshot identifiers do not establish an installation date.

A Expanded case interpretations

A.1 Specification inflation

Interpretation: specification growth outpaced the behaviour validated through product use. By 22 October the BRD had reached version 0.19 and about 37,900 words, with requirement identifiers up to FR-052 and AC-103; the PRD had reached version 0.18 and about 33,900 words, with identifiers up to FR-061 and AC-110. The backlog inherited that scale and grew to 192

issues. Cross-cutting concerns were appended to existing issues rather than sequenced as their own work: a responsible-AI section on 15 October and an AI-consumption section on 22 October. Issue #126, on pilot release gates, ended with seventeen acceptance criteria in three sections. Even the request for the smallest runnable preview arrived with thirteen.

This expansion began before the autonomous backlog loop, including after participant prototype and MVP feedback. The requirements agents sought fully specified BRDs and PRDs, named approvers and resolved ownership before building. The author requested explicit demonstrator assumptions and imperfect increments that could produce feedback. Smaller, iterative requirements documents are a plausible alternative, not an intervention tested in this case.

The pattern resembles Böckeler's small bug that became sixteen acceptance criteria [15], at the scale of a programme, and the risks of late integration described by Royce and Gall [28, 64]. Brooks locates essential difficulty in the conceptual construct rather than its representation [17]; repeating a requirement in more places need not resolve its ambiguity. The benchmark evidence points the same way: agents given only a requirements document lose global coherence [24]. Beck's metaphor fits: features consume future options [7, 9]. Extending that argument to acceptance criteria is the present author's interpretation, not Beck's wording. The early-warning signal is the ratio of specification growth to behaviour a human has validated in the same period. No replacement journey was reported complete during the loop, but a systematic count of validated behaviours was not collected; this ratio cannot be estimated from the document counts alone.

A.2 Proxy capture

In this case, artefacts made into gates appeared to displace outcomes. Three patterns motivate this interpretation.

Merged pull requests are a weak proxy for delivered value. Murphy-Hill et al. report roughly 24% more merges among adopters in Microsoft's rollout, while explicitly distinguishing merges from value [55]. Peralta et al. studied 9,799 human-reviewed agent pull requests; in their 717-PR manually inspected sample, only 35.7% of rejections reflected clear agent failures and 15.4% of merges required explicit reviewer involvement [62]. AIDev shows the scale of this measurement problem: 932,791 agent-authored pull requests from five agents across 116,211 repositories, not a count of delivered outcomes [45].

Process. When the first waves of sessions skipped the research and planning phases, the author made the presence of their artefacts a merge condition. From then on every pull request carried research, plan, change and review records; the artefacts were guaranteed, the thinking they were meant to evidence was not. Pull request #445 changed two lines of behaviour and added a 78-line evidence record that reports its own artefact completion as 100%. HVE Core treats these files as working memory. Substantial tracking output was already committed before the added merge condition; that condition formalised an existing practice rather than explaining its onset.

In the first-parent integration view, tracking-artefact additions were 10.27 times product-source additions during the loop, compared with 29.28 before it (`anc/repo_history_metrics.out.txt`). Artefact volume remained high during the loop, while its ratio to product-source additions fell. Churn alone does not establish code decay.

Evaluation. The rubric scorer, a language model reading the repository, rated implementation completeness at five out of five, citing a README and a check script, at a time when the author could not sign in to the replacement. The score and blocked sign-in occurred during the same period. It remains necessary to check whether the score covered that release and journey (Section 3.6). The judge read repository text rather than executing the intended journey. Neither its repeatability nor its agreement with observed product behaviour was measured; its score cannot substitute for that behavioural check.

Evidence boundaries. The author had instructed the agents never to fabricate approvals or

overstate evidence. The record contains explicit statements of unmet acceptance, and lexical evidence-boundary density more than tripled. These observations do not establish exhaustive compliance or measure honesty. Issue #349 records that a browser redirect "is not successful sign-in". The agents were right, and still nobody could sign in. Honest boundary statements became the output.

Goodhart and Strathern predict the first two levels [33, 67]. Beck joked that an agent seemed ready to delete tests and pretend it had finished [11]; ImpossibleBench measures test exploitation more generally, not a counted test-deletion category [75]. The METR study adds the human side: the feeling of progress is an unreliable instrument [13]. The early-warning signals are the ratio of evidence artefacts to behaviour changes, the share of merged pull requests that change what a user can do, boundary-phrase density, and any divergence between automated scores and a live demo.

A.3 Learning evaporation

Some recorded lessons appeared to fall off the loop's read path, while important design decisions survived. Maintaining the structured log and memory became difficult as the record grew. Important design decisions survived, even as some recorded lessons appeared to be lost. Before the loop, participant use of the MVP produced concrete lessons: the simplicity requirements, a deliberately public development profile for throwaway stacks, and the callback, role and staff-binding steps that made sign-in work. All were recorded, in the PRD, in a simplicity review and in infrastructure runbooks. The stated brief format carried issue text, acceptance criteria and repository conventions; actual session access to the earlier lessons is unverified. The November search finds retained Entra, registration, staff-binding and runbook references in committed research/plans, including 16 October records. This weakens a blanket disappearance claim without establishing that agents accessed or used the material. The log cites the simplicity requirements for the last time on 13 October; the development profile appears only on 8 October. The migrated deployment lacked the registration and staff bindings established on 12 October, and sign-in was blocked.

The read path grew. The knowledge-base index, which every agent is told to read first, accumulated 1,069 words of issue-specific status notes ahead of its first instruction. The index, the log, the BRD and the PRD, the documents that define the project's state and intent, total about 139,000 words. No provider-matched tokenizer or actual session-input record was available; prose token-per-word heuristics are unsuitable for these identifiers, paths and tables. Long contexts can reduce reliable retrieval even when the material fits in the context window [35, 46]; the case does not measure any session's effective attention capacity.

Instruction debt and learning evaporation can therefore coexist. A correction may remain in an expanding agent definition while the evidence explaining when it applies falls off the next session's read path. The result can be a larger instruction set with weaker access to the relevant lesson. However, unavailable guidance, accessed-but-misapplied guidance and guidance insufficient for the task are different explanations [38, Secs. 5.2–5.5]. Without actual session inputs and access/use traces, this case cannot distinguish them.

Naur explains why recording was not enough. A program's theory lives in the people who build it, and a second team with full documentation can still destroy a design's simplicity [56]. In an agent loop, every session may face an analogous reconstruction problem. The comparison with Naur is interpretive. The case provides no measure of lost understanding. Yan's advice to share context appears within an argument against multi-agent designs [71], not evidence that adding agents solves retrieval. The multi-agent failure taxonomy [18] identifies several coordination problems; it does not establish context loss as a prevalent cause in this case. HVE Core's choice to carry context in files on disk is necessary but not sufficient: the files must be on the read path, short enough to read and, ideally, executable. The early-warning signals are lessons that exist only as prose, a read path that grows faster than the code, and problems that

have to be solved twice.

Pavlenko et al.'s World Model offers a different design: preserve raw evidence, derive rebuildable views and retrieve compact, task-specific context with provenance and access controls. Updates are reviewed, and measured outcomes feed later decisions [61]. This distinguishes storing lessons from making them usable in the next decision; a small read path need not mean discarding history. This architecture offers another way to retrieve accumulated lessons. Its effect on learning evaporation in this case remains untested.

A.4 Process-gate inflation

Interpretation: repeated gates appeared disproportionately burdensome; comparative effort was not measured. Each new gate required execution and review. Within ten days the coordinator required, for every issue, the full RPI loop, a complete local gate before every push and after every correction or rebase, independent review, up to two correction cycles, board updates and evidence-based ticking of acceptance criteria. Hosted CI minutes had to be rationed, the full gate had to be interrupted by explicit exception for a two-line fix, and paid cloud validation stayed blocked even after the cost ceiling was raised.

The author perceived attention as more constrained than compute, but their relative scarcity was not measured. Reviewing eighteen merged pull requests a day in depth was judged impractical, and merging was delegated to the coordinator. The author still answered approvals and technical blockers, but stopped getting feedback from using a replacement product. RPI critiques and reviews addressed the current issue rather than the cumulative system; there was no regular phase for reflective consolidation or generalisation across completed issues. This is Bainbridge's irony in miniature [3]: the automation took the part of the work that supplied direct product-use feedback and left the participant monitoring a stream of plausible, well-documented changes. It matches the shift from producing to evaluating reported for generative AI [65], the delegation demands it creates [68]. Higher confidence in AI was associated with reduced self-reported critical scrutiny in the cited study [42], not established as its cause. Murphy's diagnosis is a plausible interpretation: throughput increased while the path to a usable outcome remained blocked [32, 54]. The bottleneck itself was not instrumented. Martin's argument for executable feedback from tests and code-analysis tools [48] motivates a proposed gate that checks product behaviour rather than inferring it from paperwork; the interview does not establish that such a gate would have prevented this failure. The early-warning signals are gate time per change, human review minutes per merged pull request, work in progress, and cost per merged pull request compared with cost per accepted outcome.

A.5 Feedback starvation

Candidate mechanism: insufficient independent, outcome-level feedback even though artefact-level validation is abundant. This differs from merely having few tests, a long feedback delay or missing context: many checks can pass while none disconfirms the claim that the intended user journey works. Outcome-level independence here means evidence obtained by exercising the outcome rather than rereading its completion record; it need not imply a different model. It does not establish independent human judgment. The loop prioritised completing the specified plan rather than shortening the path to such feedback. Feedback starvation is a central interpretive proposition, although the case cannot separate its contribution from the other mechanisms. Nothing in the coordinator kept a usable system alive: no invariant required the participant's sign-in and core workflow to remain usable after every merge. Completion was assessed through artefacts, criteria and gates. In the MVP week the feedback loop included intermittent participant use, with some changes made the same day. From 15 October the first priority was restructuring code, the working MVP was frozen, and its replacement had not become usable by the time of the case cut-off. When a usable increment was requested, the request was absorbed

by the same machinery: a label, new issues, thirteen acceptance criteria and the full set of gates.

Several older practices in Section 1.8 make feedback and working behaviour explicit: short releases and test-first development [4, 10], incremental delivery in Crystal Clear [21], tracer bullets [37], and working software over comprehensive documentation [12]. Beck's warning that fully agentic work without feedback goes off the rails [9], DORA's emphasis on small batches and user focus [25] and Murphy's list of real bottlenecks, which includes missing feedback [54], motivate this interpretation but do not independently diagnose the case. The early-warning signals are the time from the start of a cycle to the first human use of a running increment, and the age of the last increment a human has verified as usable. The case records no completed replacement journey during the ten-day loop; the exact last-use timestamp remains unverified.

B Case rival-explanation register

The main interpretation is in Section 5.2. The register preserves recorded rivals separately from untested alternatives. A limit is not grounds for ruling an explanation out. Score/deployment correspondence remains unverified (Section 3.6).

Table 1: Rival explanations with direct case records.

Explanation	Case support	Counter-evidence or limits	Treatment
Migration decision	ADR 0006 (issue #196) froze feature work in the MVP and prioritised a new layout.	Health and restart checks passed, not usability.	Migration and coordination changed together; autonomy alone is not identified.
Capstone rubric	Research and documentation are rewarded; full completeness marks coexist with blocked sign-in.	A working demonstrator is required; release/scope linkage is pending.	Possible incentive, not a measured effect.
Owner overrides and priorities	Committed RPI records, full gates and delegated merging; PR #445 required an exception.	The MVP used agents successfully; HVE Core defaults retain human stops.	Owner choices are part of the case.
Model and tool changes	Versions changed during the study.	No matched rerun under fixed versions.	Actual session configuration remains partly unknown.
CI and cloud budgets	Issue #413 moved validation locally; costs prompted a higher ceiling and reduced resources.	Merges continued without established deployed usability.	More budget is not proven to resolve delivery.
Identity and sign-in setup	Missing registration/staff bindings; issue #349 distinguishes redirect from sign-in.	Those steps worked on 12 October.	Concrete configuration obstacle, not proof of code decay or lost knowledge.

Table 2: Additional plausible rivals and unperformed discriminating checks.

Explanation	Reason considered	Counter-evidence or limits	Evidence still needed
External services and deployment drift	Model/identity configuration differs from the MVP.	No provider outage is established; health checks were narrower.	Versioned deployment/configuration and dependency-health records for the journey.
Scope expansion	RAI and consumption criteria added to issues; further resident-preview obligations.	Expansion predates autonomy; some controls are justified.	Original/added scope, sequencing and time-to-outcome comparison.
Inadequate acceptance tests	Checks passed without completed real-user sign-in; scorer read artefacts.	Bounded, honest tests need not be poor tests.	Capability-to-task-oracle mapping and execution evidence.
Rate and context limits	Parallel sessions and a large read path.	No runtime rate-limit, truncation or attention dataset analysed.	Actual inputs and blocked/retried request events.
Specification ambiguity	Selective review of long, cross-referenced requirements.	Important domain/consent decisions survived; ambiguity not independently coded.	Independent interpretation and conflicting acceptance examples.
Ordinary prototype debt	Rapid MVP followed by restructuring and rebuilding.	MVP worked intermittently; no comparable human-led migration.	Failure analysis distinguishing inherited debt from introduced faults.

C Candidate-mechanism evidence audits

These are operational boundaries for this paper, not validated necessary and sufficient conditions for all agentic development. The indicators refer to Table 1 and the case chronology. Experimental implications state what evidence would discriminate an explanation; the tagged experiment does not manipulate the five candidates separately.

C.1 Specification inflation

Element	Audit
Definition	Requirements and acceptance obligations expand faster than independently validated behaviour, without evidence that the added scope is needed for the next usable increment. Length alone is insufficient.
Case indicators	O3: BRD/PRD growth, 192 issues and cross-cutting additions; the smallest resident preview had thirteen criteria.
Counter-evidence	Growth began before autonomy and sometimes followed the owner's feedback; regulatory and consent controls can justify detail.
Rival explanations	Genuine scope expansion, specification ambiguity, capstone incentives and an ambitious migration rather than agent-driven inflation.
Experimental implication	Compare smaller requirements horizons at matched scope and resources. The current fixed-suite experiment cannot identify this mechanism, even if consolidation helps.

C.2 Proxy capture

Element	Audit
Definition	A completion proxy becomes a decision criterion and remains positive when an independent test of the intended outcome is negative. Proxy use alone does not establish capture.
Case indicators	O2–O3: a 5/5 score and blocked sign-in coexist in the chronology; release/scope linkage is pending. Mandatory artefact presence and the evidence record for PR #445 are separate indicators.
Counter-evidence	Agents explicitly reported unmet acceptance; neither fabricated approval nor deliberate deception is established.
Rival explanations	An inadequate scorer or acceptance oracle, different evaluation targets, and a deployment failure invisible to repository review.
Experimental implication	Disagreement between visible completion and hidden behavioural tests would expose proxy limits. It would not by itself establish intentional test gaming or explain why a gate was chosen.

C.3 Learning evaporation

Element	Audit
Definition	A recorded lesson relevant to a later decision is absent from, unusable on, or not applied through that decision's effective context path, and the associated problem recurs. A missing citation alone is insufficient.
Case indicators	O4: simplicity references cease; working sign-in setup is not preserved; state and intent documents reach about 139,000 words.
Counter-evidence	Important domain decisions survived; the lesson could have been known but deferred. Actual session attention was not measured.
Rival explanations	Deliberate migration sequencing, deployment-specific configuration, context limits, or conflicting requirements rather than lost knowledge.
Experimental implication	Trace actual inputs and repeated faults. B–C cannot isolate memory quality because DESIGN and REVIEW notes and acting permissions differ. A matched-context study would be needed.

C.4 Process-gate inflation

Element	Audit
Definition	Additional procedural obligations increase per-change burden without demonstrated proportional improvement in the targeted outcome. More gates alone are not evidence that their net benefit is negative.
Case indicators	O5: full RPI and validation on each correction/rebase; hosted-minute rationing; explicit exception for PR #445.
Counter-evidence	Checks caught bounded defects and enforced evidence integrity. Gate duration and human review effort were not measured per PR.
Rival explanations	Legitimate safety and privacy requirements, budget constraints, new deployment complexity and owner-selected risk tolerance.
Experimental implication	Measure gate time and accepted outcomes under matched risk. The pilot's resource description concerns the package, not the effect of removing this repository's required gates.

C.5 Feedback starvation

Element	Audit
Definition	Independent outcome-level feedback is insufficient to correct delivery decisions despite abundant artefact-level validation. The defining gap is between the evaluated proxy and intended outcome, not test count alone.
Case indicators	O1–O2 and O5: working MVP frozen; replacement unusable; passing checks and continued approvals did not restore a completed user task.
Counter-evidence	Human oversight continued; agents surfaced blockers; engineering checks provided real, though narrower, feedback.
Rival explanations	Identity and service configuration, migration priority, incomplete acceptance coverage, prototype debt or insufficient resources.
Experimental implication	Hidden retention tests and CR1 supply behavioural feedback to the evaluator. The experiment does not test live deployment recovery, product selection or a human-use checkpoint cadence.

D Measures and lexicon

Log entries. The knowledge-base log (`docs/reference/log.md`) was split into entries at each second-level heading that begins with a date. An entry belongs to the date in its heading. Word counts use whitespace tokenisation of the entry, heading included.

Evidence-boundary phrases. Each entry was searched, case-insensitively and on word boundaries, for these patterns: *unmet*; *not accepted*; *not inferred*; *is inferred*; *remain(s) open*; *remain(s) unproved*; *remain(s) unverified*; *not claimed*; *not established*; and *no* followed, within forty letters, spaces, commas, slashes or hyphens, by *is* or *was* and then *claimed* or *inferred*. Density is matches per 1,000 words.

Reference tracing. An entry cites the simplicity requirements if it contains an identifier from SA-001 to SA-010 or the word *simplicity*. It cites the development profile if it contains *deploymentProfile*, *development profile* or *allowPublicNetworkAccess*.

Repository counts. Issue and pull-request counts come from the GitHub search API on 24 October 2026 (for example, `is:pr is:merged merged:>=2026-10-15`). Document sizes are whitespace word counts of the files at commit `29c6b51`.

Proposed measures. *Time to first human use*: elapsed time from the start of a delivery cycle to the first time a person uses the running increment for its purpose. *Age of the last usable increment*: time since a person last verified that the deployed system could be used for its main journey. *Behaviour-changing merge ratio*: the share of merged pull requests that change what a user can observe in the running increment. *Cost per accepted outcome*: total model, tool and cloud cost divided by the number of outcomes a human accepted in the same period.

E Consolidation step prompt

Verbatim frozen v4 prompt from execution source `e8707ecc8965d3f81f5142a2603042bfca1aab15`, not the amended study's approved prompt. The archived source is byte-identical, SHA-256 `75f646edbe04e1e3403b092b13edd6b66615a3362e67ad49744d229cdcb94f29`. Display wrapping does not alter its text. Braces mark runner parameters.

```
<!-- markdownlint-disable-file -->
# Consolidation step prompt (arm B)
```

```
Do not add or change features. Your job is to leave the code better than you found it without
↪  changing its behaviour.

1. Run `make check`, then read reports/quality.md (functions above complexity 10, duplicated
↪  blocks, lint warnings, file sizes).
2. Review the last feature's diff (`git diff feature-start feature-end`) and the hotspots
↪  against this checklist:
   * duplicated logic;
   * long or complex functions;
   * unclear names;
   * dead code;
   * responsibilities in the wrong place, or rules from one feature leaking into another;
   * tests that check implementation details instead of behaviour.
3. Make small, behaviour-preserving changes. Apply every change through `./tcr "<message>"`: it
↪  runs all tests and commits if they pass, or reverts if they fail.
4. Never modify or delete files in tests/examples/. You may add tests that pin current
↪  behaviour before you refactor.
5. Rewrite DESIGN.md in at most 300 words: the current structure, where each business rule
↪  lives, and known debts. Replace its content; do not append.
6. Stop when the quality budget is met (no function above complexity 10, duplicated lines below
↪  3%, zero warnings in the configured lint rules) or when your time budget
↪  ({consolidation_budget}) is used.
7. Finish with a summary of at most five lines: what you changed and what debt remains.
```

F Deferred intervention measures

The study excludes intervention-rate outcomes but retains help and administrative records for eligibility and cost. These definitions are proposed future measures, not collected results. HG-DAgger and ThriftyDAgger supply cross-domain measurement precedents, not coding-agent effectiveness evidence [36, 39].

For run r, n_r is attempted feature cycles, including assigned post-steps and failed cycles, and h_{rj} qualifying corrective-help events. Observed human intervention rate would be

$$\mathrm{HIR}_r = \frac{1}{n_r} \sum_{j=1}^{n_r} \mathbf{1}\{h_{rj} > 0\}. \tag{1}$$

Event counts and active human minutes are separate; repeated help in one cycle counts once for incidence. With $d_{rj} = 1$ for an explicit help request or a block needing corrective human action,

$$\mathrm{IDR}_r = \frac{1}{n_r} \sum_{j=1}^{n_r} d_{rj}. \tag{2}$$

Requests and supplied help differ. A timeout alone is not a help request; zero attempts gives undefined rates. A future annotation protocol needs event/cycle/phase locators, coverage checks and adjudication; absent records cannot be zero. Routine launches, planned approvals and safety stops differ from corrective rescue. Low intervention must not reward silent failure or discourage necessary human judgement.

G Historical freeze provenance

The historical v4 execution source was `e8707ecc8965d3f81f5142a2603042bfca1aab15`, with manifest-only commit `03c50856838b539780d57216af2a4cf6744187d7`. Its attached `osf-v4-2026-10-31.tar.gz` checksum was `1a42fac40a4f1aa82df59c8c3f7f8d6dce37d22502c5952c398362ac63d6bd65`. The runtime image was `sha256:090220b0b172ec205f1ea473d7c3a223eec5d7d619eb21acdcf15c9289287fa3`. These identify historical inputs, not a freeze or approval

of the five-arm amendment. The original registration at https://osf.io/ywsrv/ is owner-reported approved after submission on 28 October; the inspected v4 prospective amendment flag was false and its completion date null. New approval and actual prospective recording cannot be inferred or backdated.

References

[1] Shyam Agarwal, Anmol Singhal, Andrew Macvean, Bogdan Vasilescu, and Travis Breaux. Spec-driven development as prescribed and as practiced, 2026.

[2] Alberto Bacchelli and Christian Bird. Expectations, outcomes, and challenges of modern code review. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, pages 712–721. IEEE, 2013. doi: 10.1109/ICSE.2013.6606617.

[3] Lisanne Bainbridge. Ironies of automation. *Automatica*, 19(6):775–779, 1983. doi: 10.1016/0005-1098(83)90046-8.

[4] Kent Beck. *Test-Driven Development: By Example*. Addison-Wesley, 2002. ISBN 978-0-321-14653-3.

[5] Kent Beck. *Tidy First? A Personal Exercise in Empirical Software Design*. O'Reilly Media, 2023. ISBN 978-1-098-15124-9.

[6] Kent Beck. Augmented coding: Beyond the vibes. Software Design: Tidy First? (newsletter), 25 June 2025, https://tidyfirst.substack.com/p/augmented-coding-beyond-the-vibes, 2025. Accessed 1 November 2026; brief excerpt.

[7] Kent Beck. Why does development slow? Software Design: Tidy First?, 19 November 2025; author's summary at https://kentbeck.com/summaries/why-does-development-slow/, 2025. Accessed 1 November 2026; author's complete summary, not the full essay.

[8] Kent Beck. Mathematicians, here's a way to think about your existential crisis. Software Design: Tidy First? (newsletter), 29 September 2026, https://newsletter.kentbeck.com/p/mathematicians-heres-a-way-to-think, 2026. Accessed 1 November 2026; brief excerpt.

[9] Kent Beck. Software engineering in the age of AI. Talk at Prodacity 2026, Nashville, TN, 25–27 August 2026. Video https://www.youtube.com/watch?v=F8fBgDCf2Y4; transcript https://www.rise8.us/videos/kent-beck-software-engineering-in-the-age-of-ai, 2026. Transcript accessed 1 November 2026; video-page HTML only, video not inspected.

[10] Kent Beck and Cynthia Andres. *Extreme Programming Explained: Embrace Change*. Addison-Wesley, 2nd edition, 2004. ISBN 978-0-321-27865-4.

[11] Kent Beck and Gergely Orosz. TDD, AI agents and coding with Kent Beck. The Pragmatic Engineer podcast, 11 June 2025, https://www.youtube.com/watch?v=aSXaxOdVtAQ, 2025. Video-page retrieval attempted 1 November 2026; HTML only, video/transcript not inspected.

[12] Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland, and Dave Thomas. Manifesto for agile software development. https://agilemanifesto.org/, 2001. Accessed 1 November 2026; manifesto excerpt.

[13] Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein. Measuring the impact of early-2025 AI on experienced open-source developer productivity, 2025.

[14] Terese Besker, Antonio Martini, and Jan Bosch. Technical debt cripples software developer productivity: A longitudinal study on developers' daily software development work. In *Proceedings of the International Conference on Technical Debt (TechDebt '18)*, pages 105–114. ACM, 2018. doi: 10.1145/3194164.3194178.

[15] Birgitta Böckeler. Understanding spec-driven-development: Kiro, spec-kit, and Tessl. martinfowler.com, 15 October 2025, https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html, 2025. Accessed 1 November 2026; brief excerpt.

[16] Markus Borg, Dave Hewett, Nadim Hagatulah, Noric Couderc, Emma Söderberg, Donald Graham, Uttam Kini, and Dave Farley. Echoes of AI: Investigating the downstream effects of AI assistants on software maintainability. *Empirical Software Engineering*, 31(6):161, 2026. doi: 10.1007/s10664-026-10889-1.

[17] Frederick P. Brooks, Jr. No silver bullet: Essence and accidents of software engineering. *Computer*, 20(4):10–19, 1987. doi: 10.1109/MC.1987.1663532.

[18] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025), Datasets and Benchmarks Track*, 2025.

[19] Worawalan Chatlatanagulchai, Hao Li, Yutaro Kashiwa, Brittany Reid, Kundjanasith Thonglek, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, Bram Adams, Ahmed E. Hassan, and Hajimu Iida. Agent READMEs: An empirical study of context files for agentic coding. *ACM Transactions on Software Engineering and Methodology*, 2026. doi: 10.1145/3840295. URL https://arxiv.org/abs/2511.12884v2. Inspected text: version 2, 9 August 2026; first posted 2025.

[20] Jialong Chen, Xander Xu, Hu Wei, Chuan Chen, and Bing Zhao. SWE-CI: Evaluating agent capabilities in maintaining codebases via continuous integration, 2026.

[21] Alistair Cockburn. *Crystal Clear: A Human-Powered Methodology for Small Teams*. Addison-Wesley, 2004.

[22] Ward Cunningham. The WyCash portfolio management system. In *Addendum to the Proceedings of OOPSLA '92 (Experience Report)*, pages 29–30. ACM, 1992. doi: 10.1145/157709.157715.

[23] Den Delimarsky. Spec-driven development with AI: Get started with a new open source toolkit. The GitHub Blog, 2 September 2025, https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/, 2025. Accessed 1 November 2026; brief excerpt.

[24] Jingzhe Ding, Shengda Long, Changxin Pu, et al. NL2Repo-Bench: Towards long-horizon repository generation evaluation of coding agents, 2025.

[25] DORA. State of AI-assisted software development 2025. Technical report, Google Cloud, 2025. URL https://dora.dev/dora-report-2025. Accessed 1 November 2026; landing-page metadata, not the report's full text.

[26] Stephen G. Eick, Todd L. Graves, Alan F. Karr, J. S. Marron, and Audris Mockus. Does code decay? assessing the evidence from change management data. *IEEE Transactions on Software Engineering*, 27(1):1–12, 2001. doi: 10.1109/32.895984.

[27] Martin Fowler. Design stamina hypothesis. martinfowler.com, 20 June 2007, https://martinfowler.com/bliki/DesignStaminaHypothesis.html, 2007. Accessed 1 November 2026; full web essay.

[28] John Gall. *General Systemantics: An Essay on How Systems Work, and Especially How They Fail*. General Systemantics Press, Ann Arbor, MI, 1975.

[29] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995. ISBN 978-0-201-63361-0. URL https://www.informit.com/store/design-patterns-elements-of-reusable-object-oriented-9780201633610. Publisher preface and Adapter excerpt inspected 29 October 2026; copyright year 1995, preface dated August 1994; Accessed 1 November 2026; publisher description, not the full book.

[30] Dhruv Gautam, Spandan Garg, Jinu Jang, Neel Sundaresan, and Roshanak Zilouchian Moghaddam. RefactorBench: Evaluating stateful reasoning in language agents through code. In *International Conference on Learning Representations (ICLR)*, 2025.

[31] Thibaud Gloaguen, Niels Mündler-Sasahara, Mark Niklas Müller, Veselin Raychev, and Martin Vechev. Evaluating AGENTS.md: Are repository-level context files helpful for coding agents?, 2026. URL https://arxiv.org/abs/2602.11988v3. Version 3, 29 September 2026.

[32] Eliyahu M. Goldratt and Jeff Cox. *The Goal: A Process of Ongoing Improvement*. North River Press, 1984.

[33] Charles A. E. Goodhart. Problems of monetary management: The U.K. experience. In *Papers in Monetary Economics*, volume I, pages 1–20. Reserve Bank of Australia, Sydney, 1975.

[34] Israel Herraiz, Daniel Rodriguez, Gregorio Robles, and Jesus M. Gonzalez-Barahona. The evolution of the laws of software evolution: A discussion based on a systematic literature review. *ACM Computing Surveys*, 46(2):28, 2013. doi: 10.1145/2543581.2543595.

[35] Kelly Hong, Anton Troynikov, and Jeff Huber. Context rot: How increasing input tokens impacts LLM performance. Technical report, Chroma, 2025. URL https://trychroma.com/research/context-rot. Accessed 1 November 2026; brief excerpt.

[36] Ryan Hoque, Ashwin Balakrishna, Ellen Novoseller, Albert Wilcox, Daniel S. Brown, and Ken Goldberg. ThriftyDAgger: Budget-aware novelty and risk gating for interactive imitation learning. In *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, pages 598–608. PMLR, 2022. URL https://proceedings.mlr.press/v164/hoque22a.html.

[37] Andrew Hunt and David Thomas. *The Pragmatic Programmer: From Journeyman to Master*. Addison-Wesley, 1999. ISBN 0-201-61622-X.

[38] Zhiyuan Jiang, Fangrui Huang, Hanwen Xing, Xander Wu, Yipeng Gao, Rui Cao, Mengdi Wang, Shilong Liu, and Yijiang Li. Demystifying agent skills: Why they work–until they don't, 2026. URL https://arxiv.org/html/2608.14036v1. Preprint, version 1; accessed 29 October 2026.

[39] Michael Kelly, Chelsea Sidrane, Katherine Driggs-Campbell, and Mykel J. Kochenderfer. HG-DAgger: Interactive imitation learning with human experts. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 8077–8083, 2019. First posted 2018.

[40] Aayush Kumar, Yasharth Bajpai, Sumit Gulwani, Gustavo Soares, and Emerson Murphy-Hill. Why AI agents still need you: Findings from developer-agent collaborations in the wild. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 432–444, 2025. Version of 9 October 2025.

[41] Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. LLMs get lost in multi-turn conversation. In *International Conference on Learning Representations (ICLR)*, 2026. Version 1, 9 May 2025.

[42] Hao-Ping Lee, Advait Sarkar, Lev Tankelevitch, Ian Drosos, Sean Rintel, Richard Banks, and Nicholas Wilson. The impact of generative AI on critical thinking: Self-reported reductions in cognitive effort and confidence effects from a survey of knowledge workers. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pages 1–22. ACM, 2025. doi: 10.1145/3706598.3713778.

[43] M. M. Lehman, J. F. Ramil, P. D. Wernick, D. E. Perry, and W. M. Turski. Metrics and laws of software evolution—the nineties view. In *Proceedings of the 4th International Software Metrics Symposium (METRICS '97)*. IEEE, 1997. doi: 10.1109/METRIC.1997.637156.

[44] Meir M. Lehman. Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9):1060–1076, 1980. doi: 10.1109/PROC.1980.11805.

[45] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. AIDev: Studying AI coding agents on GitHub. In *Proceedings of the International Conference on Mining Software Repositories (MSR)*, 2026. doi: 10.1145/3793302.3797249.

[46] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl_a_00638.

[47] Yanuo Ma, Ben Kereopa-Yorke, and Ben Schultz. Building to the test: Coding agents deliver what you check, not what you requested, 2026. URL https://arxiv.org/abs/2606.28430v1. Preprint, 26 June 2026.

[48] Robert C. Martin. Why hardcore technical focus still matters big time. Product Engineers podcast, 29 September 2026, https://productengineers.com/episodes/season2/why-hardcore-technical-focus-still-matters-big-time-uncle-bob/, 2026. Accessed 1 November 2026; episode description, not the complete audio.

[49] Sanket Mhatre, Yasharth Bajpai, Sumit Gulwani, Emerson Murphy-Hill, and Gustavo Soares. SWE-Sharp-Bench: A reproducible benchmark for C# software engineering tasks. In *International Conference on AI-powered Software (AIware)*, 2025. Accessed version of 5 November 2025.

[50] Microsoft. HVE Core: Hypervelocity engineering core. GitHub repository and documentation, tag plugins-v3.3.106, https://github.com/microsoft/hve-core/tree/plugins-v3.3.106; https://microsoft.github.io/hve-core/, 2026. Accessed 24 October 2026; Tag recheck attempted 1 November 2026; organization SAML denied access, no bypass attempted.

[51] Microsoft Research. Systematic debugging for AI agents: Introducing the AgentRx framework. First-party research blog, 12 March 2026, 2026. URL https://www.microsoft.com/en-us/research/blog/systematic-debugging-for-ai-agents-introducing-the-agentrx-framework/. Accessed 27 October 2026; Accessed 1 November 2026; first-party summary excerpt.

[52] Microsoft Research. SkillOpt: Agent skills as trainable parameters. First-party research blog, 30 June 2026, 2026. URL https://www.microsoft.com/en-us/research/blog/skillopt-agent-skills-as-trainable-parameters/. Accessed 27 October 2026; Accessed 1 November 2026; first-party summary excerpt.

[53] Seyedmoein Mohsenimofidi, Matthias Galster, Christoph Treude, and Sebastian Baltes. Context engineering for AI agents in open-source software. In *Proceedings of the 23rd International Conference on Mining Software Repositories*, pages 194–198, 2026. doi: 10.1145/3793302.3793350. URL https://arxiv.org/abs/2510.21413v4. Version 4, 5 February 2026; first posted 2025.

[54] Andrew Murphy. If you thought the speed of writing code was your problem – you have bigger problems. Blog post, 17 March 2026, https://andrewmurphy.io/blog/if-you-thought-the-speed-of-writing-code-was-your-problem-you-have-bigger-problems, 2026. Accessed 1 November 2026; page metadata/brief excerpt.

[55] Emerson Murphy-Hill, Jenna Butler, and Alexandra Savelieva. Adoption and impact of command-line AI coding agents: A study of Microsoft's early 2026 rollout of Claude Code and GitHub Copilot CLI, 2026.

[56] Peter Naur. Programming as theory building. *Microprocessing and Microprogramming*, 15 (5):253–261, 1985. doi: 10.1016/0165-6074(85)90032-8.

[57] Matt Nigh. How to write a great agents.md: Lessons from over 2,500 repositories. The GitHub Blog, 19 November 2025, updated 25 November 2025, 2025. URL https://github.blog/ai-and-ml/github-copilot/how-to-write-a-great-agents-md-lessons-from-over-2500-repositories/. Accessed 1 November 2026; brief excerpt.

[58] Gabriel Orlanski, Devjeet Roy, Alexander Yun, Changho Shin, Alex Gu, Albert Ge, Dyah Adila, Nicholas Roberts, Frederic Sala, and Aws Albarghouthi. SlopCodeBench: Benchmarking how coding agents degrade over long-horizon iterative tasks, 2026. Inspected preprint version 2; conference status not independently verified.

[59] Ruwei Pan, John Zhang, Linhao Wu, Yakun Zhang, Hongyu Zhang, and Lu Zhang. Do modifications made by coding agents make subsequent software maintenance harder?, 2026. Preprint, 10 October 2026.

[60] David Lorge Parnas. Software aging. In *Proceedings of the 16th International Conference on Software Engineering (ICSE '94)*, pages 279–287. IEEE, 1994. doi: 10.1109/ICSE.1994.296790.

[61] Anna Pavlenko, Bogdan Crivat, Brandon Haynes, Carlo Curino, Fotis Psallidas, Jaro Slawinski, Johannes Freischuetz, Laura Pereira Sanchez, Markus Weimer, Mathieu Demarne, Matthias Jasny, Mauttik Gandhi, Max Bovykin, Mirco Milletari, Purbasha Ghosh, Qiushi Bai, Raghu Ramakrishnan, Rahul Pandita, Sergiy Matusevich, Shivaram Venkataraman, Subru Krishnan, Md. Tareq Mahmood, Tiemo Bang, Venkatesh Emani, Xuan Zhao, and Yiwen Zhu. Towards an AI software factory for data systems, 2026. URL https://arxiv.org/abs/2609.36323v1. Version 1, submitted 28 September 2026; accessed 25 October 2026.

[62] Sien Reeve O. Peralta, Fumika Hoshi, Hironori Washizaki, Naoyasu Ubayashi, Inase Kondo, Yoshiki Higo, Hiroki Mukai, Norihiro Yoshida, Kazuki Kusama, Hidetake Tanaka, and Youmei Fan. Why are agentic pull requests merged or rejected? an empirical study. In *Proceedings of the 23rd International Conference on Mining Software Repositories*, pages 787–791, 2026. doi: 10.1145/3793302.3793575.

[63] Prithvi Rajasekaran, Ethan Dixon, Carly Ryan, and Jeremy Hadfield. Effective context engineering for AI agents. Anthropic, 29 September 2025, 2025. URL https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents. Accessed 1 November 2026; brief excerpt at redirected official page.

[64] Winston W. Royce. Managing the development of large software systems. In *Proceedings of IEEE WESCON*, pages 1–9, August 1970.

[65] Auste Simkute, Lev Tankelevitch, Viktor Kewenig, Ava Elizabeth Scott, Abigail Sellen, and Sean Rintel. Ironies of generative AI: Understanding and mitigating productivity loss in human-AI interaction. *International Journal of Human–Computer Interaction*, 41(5): 2898–2919, 2025. doi: 10.1080/10447318.2024.2405782.

[66] Akshit Sinha, Arvindh Arun, Shashwat Goel, Steffen Staab, and Jonas Geiping. The illusion of diminishing returns: Measuring long horizon execution in LLMs. In *International Conference on Learning Representations (ICLR)*, 2026. Version 3, 13 March 2026.

[67] Marilyn Strathern. 'improving ratings': Audit in the British university system. *European Review*, 5(3):305–321, 1997.

[68] Lev Tankelevitch, Viktor Kewenig, Auste Simkute, Ava Elizabeth Scott, Advait Sarkar, Abigail Sellen, and Sean Rintel. The metacognitive demands and opportunities of generative AI. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, pages 1–24. ACM, 2024. doi: 10.1145/3613904.3642902.

[69] Adam Tornhill and Markus Borg. Code red: The business impact of code quality – a quantitative study of 39 proprietary production codebases. In *Proceedings of the International Conference on Technical Debt (TechDebt '22)*, pages 11–20. ACM, 2022. doi: 10.1145/3524843.3528091.

[70] Muhammad Waseem, Aakash Ahmad, Kai-Kristian Kemell, Jussi Rasku, Sami Lahti, Kalle Mäkelä, and Pekka Abrahamsson. Vibe coding in practice: Flow, technical debt, and guidelines for sustainable use, 2025.

[71] Walden Yan. Don't build multi-agents. Cognition blog, 12 June 2025, https://cognition.com/blog/dont-build-multi-agents, 2025. Accessed 1 November 2026; context-sharing principles within the argument against multi-agent designs.

[72] Chenyang Yang, Hao He, Ti-Chung Cheng, Jenna Butler, Emerson Murphy-Hill, Carmen Badea, and Christian Bird. (when) should a software factory be lights-off? benchmarking code review agents on bug detection, 2026. URL https://arxiv.org/abs/2610.13668v1. Preprint, 9 October 2026.

[73] Haoran Yu, Lifei Liu, Xiaochong Jiang, Yuwen Jia, Su Wang, Pin Qian, and Yihang Chen. Habituation at the gate: Rising approval and declining scrutiny in human review of AI agent code. KDD 2026 Workshop on Agentic Software Engineering, 2026.

[74] Muru Zhang, Ofir Press, William Merrill, Alisa Liu, and Noah A. Smith. How language model hallucinations can snowball. In *Proceedings of the 41st International Conference on*

Machine Learning, volume 235 of *Proceedings of Machine Learning Research*, pages 59670–59684. PMLR, 2024. URL https://proceedings.mlr.press/v235/zhang24ay.html. Inspected original text: arXiv version 1, 22 May 2023; publication metadata verified 1 November 2026.

[75] Ziqian Zhong, Aditi Raghunathan, and Nicholas Carlini. ImpossibleBench: Measuring LLMs' propensity of exploiting test cases. In *International Conference on Learning Representations (ICLR)*, 2026.